

KVPR: 基于 I/O 感知 KV 缓存部分重计算的高效 LLM 推理

Chaoyi Jiang*, Lei Gao*, Hossein Entezari Zarch, Murali Annavarm

University of Southern California

{chaoyij, leig, entezari, annavara}@usc.edu

摘要

大语言模型 (LLM) 推理计算开销很高。为降低自回归解码成本，系统通常使用 Key-Value (KV) 缓存保存中间激活，从而显著降低 token 生成的计算开销。然而，KV 缓存所需内存会快速增长，常常超过 GPU 显存容量。一种成本更低的替代方案是将 KV 缓存卸载到 CPU 内存，以缓解 GPU 显存压力，但这又把瓶颈转移到 CPU 与 GPU 之间 PCIe 连接的有限带宽上。现有方法尝试通过重叠 GPU 计算与 I/O，或采用 CPU-GPU 异构执行来缓解这些问题，但它们会受到过量数据移动和 CPU 能力依赖的限制。随着 KV 缓存增大和/或 GPU 计算能力增强，完全隐藏 PCIe 通信延迟会变得越来越困难。本文提出 KVPR，一种高效的 I/O 感知 LLM 推理方法：CPU 先传输一部分激活值，GPU 基于这些激活值开始重计算 KV 缓存；在 GPU 重计算部分 KV 缓存的同时，CPU 并发传输剩余 KV 缓存。该方法通过重叠 GPU 重计算与 KV 缓存传输，最小化 GPU 空闲时间并最大化推理性能。KVPR 通过集成 profiler、scheduler 和 runtime 实现全自动化：profiler 利用输入特征和系统硬件信息，scheduler 优化计算与通信工作负载划分，runtime 高效执行生成的执行计划。实验结果表明，相比当前最先进方法，KVPR 在解码阶段最高降低 35.8% 延迟，并提升 46.2% 吞吐量。代码已开源：<https://github.com/chaoyij/KVPR>。

1 引言

近年来，大型语言模型 (LLM) 取得了显著进展，展示了它们为机器翻译等多种应用提供支持的能力 (Zhu et al., 2024)，总结 (OpenAI et al., 2024)、创意内容生成 (Gemini Team, 2024)，以及个性化推荐 (Geng et al., 2022)。实时应用程序，包括对话代理和实时翻译 (Li et al., 2023)，依靠低延迟来提供无缝的用户交互，而大规模部署则需要高吞吐量来支持并发用户并高效处理大量数据 (Kwon et al., 2023)。

键值 (KV) 缓存对于 LLM 的自回归解码至关重要，因为它存储注意力机制中早期步骤的中间键和值激活。通过消除为每个生成的令牌重计算这些激活的需要，这降低了生成每个令牌从二次到线性的计算复杂性。然而，这是有代价的：KV 缓存的大小随着批量大小、序列长度和模型大小线性增长，导致大量的内存需求 (Wan et al., 2024)。

GPU 内存虽然针对计算单元的高带宽访问进行了优化，但本质上是有限的，并且通常不足以处理庞大且不断增长的 KV 缓存。解决这一限制的一种经济高效的方法是将 KV 缓存卸载到更便宜且充足的 CPU 内存，并且可以进一步卸载到硬盘和网络存储 (Liu et al., 2024a)。虽然卸载减少了 GPU 内存压力，但它引入了一个新的瓶颈：缓慢的 PCIe 总线成为将 KV 缓存从 CPU 转移到 GPU 进行计算时的限制因素。由于 PCIe 传输时间较长，导致整体解码延迟增加，令牌生成吞吐量下降，阻碍了系统的整体推理效率 (Zhao et al., 2024a)。

为了评估通信开销的影响，我们建立了

*These authors contributed equally.

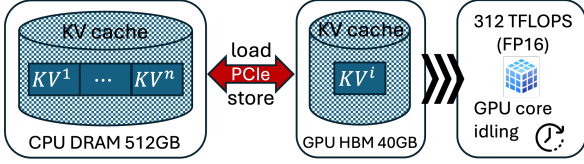


图 1: 配备 A100 GPU 的 LLM 推理系统。

Model	Hidden Dim	KV Cache (MB)	PCIe Latency (ms)	Comp. Latency (ms)
OPT-6.7B	4,096	512	15.6	0.3509
OPT-13B	5,120	640	19.5	0.4388
OPT-30B	7,168	896	27.3	0.6143

表 1: 图 1 所示系统中, 不同 KV 缓存大小对应的 PCIe 延迟和计算延迟。

如图 1 所示的 LLM 推理服务系统, 使用 NVIDIA A100 GPU。CPU DRAM 和 GPU HBM 之间的数据传输通过 PCIe 4.0 x16 完成, 带宽为 32 GB/s。表 1 展示了不同模型的隐藏维度、KV 缓存大小、PCIe 传输时间和 GPU 计算延迟。请注意, 端到端解码延迟还包含前馈层等其他组件; 为清楚起见, 这些组件未在表中列出。我们使用 FP16 精度、批量大小 32、序列长度 1024。结果表明, PCIe 延迟比 KV 缓存重计算延迟高一个数量级以上。因此, 在 KV 缓存存储于 CPU DRAM 的系统中, 较长的数据传输时间会造成 GPU 空闲, 损害推理效率。

为了解决 PCIe 延迟和带宽问题, FlexGen (Sheng et al., 2023) 和 PipeSwitch (Bai et al., 2020) 尝试将当前层的 GPU 计算与下一层的 KV 缓存加载重叠。然而, 这种重叠的有效性受到耗时最长任务的限制。在大多数系统中, PCIe 传输时间超过 GPU 计算延迟, 尤其是在批量和上下文较大的情况下。因此, 完全用 GPU 计算隐藏 PCIe 传输时间并不现实。Fast-Decode (He and Zhai, 2024) 建议直接在 CPU 上计算注意力分数, 与 GPU 相比, CPU 可以更快地访问 KV 缓存。同样, HeteGen (Zhao et al., 2024a), TwinPilots (Yu et al., 2024), 和 Park and Egger 采用 CPU-GPU 异构执行, 通过在 CPU 上执行计算来隐藏数据传输开销。然而, 正如我们稍后的结果所证明的, 这种方

法给 CPU 带来了负担, 无法满足连接到 CPU 主机的多个 GPU 的 KV 缓存计算需求, 从而限制了可扩展性。

本文提出 KVPR, 一种面向高效 LLM 推理的新方法, 用于平衡 GPU 计算与 PCIe 带宽之间的取舍。CPU 不再把整个 KV 缓存从 CPU 传输到 GPU 后再计算注意力分数, 而是先将更小的部分激活集传输到 GPU, 这些激活可用于生成一部分 KV 缓存。随后 GPU 根据输入激活重计算部分 KV 缓存; 与此同时, CPU 通过 PCIe 传输剩余 KV 缓存。KVPR 在不引入近似的前提下计算精确注意力分数, 并最大限度减少 GPU 空闲时间, 从而改善整体延迟和吞吐量。

KVPR 通过确定需要重计算的激活比例, 使 PCIe 传输时间与 GPU 重计算时间实现近乎完美的重叠。KVPR 可自动确定重计算与通信之间的切分: profiler 收集系统硬件信息, scheduler 将问题表述为线性规划以确定最优切分点, runtime 管理两个设备上的内存分配并协调数据传输。实验结果表明, 根据不同工作负载, KVPR 可显著降低推理延迟或提高吞吐量。总之, 我们的贡献如下:

- 我们提出了一种高效的 CPU-GPU I/O 感知 LLM 推理方法, 该方法利用 KV 缓存部分重计算和异步 KV 缓存传输 (重叠计算和通信) 来解决从 CPU 内存加载大型 KV 缓存的系统瓶颈。
- 我们开发了一个基于线性整数规划的框架, 可实现最佳的计算通信分配。
- 实验结果表明, KVPR 相比当前最先进方法最高可降低 35.8% 延迟, 并提升 46.2% 吞吐量。

2 背景

LLM 推理过程。 仅解码器 LLM 的推理过程采用自回归方法, 顺序生成 token。它由两个阶段组成: **预填充阶段**和 **解码阶段**。在预填充阶段, 第 i 个解码器层的输入表示为 $X^i \in \mathbb{R}^{b \times s \times h}$, 其中 $i \in \{1, \dots, n\}$, b 是批量

大小, s 是提示长度, h 是输入嵌入维度。多头注意力 (MHA) 块通过对 X^i 做线性投影来计算查询 (Q)、键 (K) 和值 (V):

$$Q^i = X^i \cdot W_Q^i, \quad K^i = X^i \cdot W_K^i, \quad V^i = X^i \cdot W_V^i, \quad (1)$$

其中 $W_Q^i, W_K^i, W_V^i \in \mathbb{R}^{h \times h}$ 是投影矩阵。生成的 K^i 和 V^i 存储在 KV 缓存中。

MHA 中的自注意力分数计算如下:

$$Z^i = \text{softmax} \left(\frac{Q^i (K^i)^T}{\sqrt{d_{\text{head}}}} \right) \cdot V^i, \quad (2)$$

其中 d_{head} 表示每个注意力头的维度。最后, 将注意力分数与线性投影一起应用以产生 MHA 块的输出:

$$O^i = Z^i \cdot W_O^i, \quad (3)$$

其中 $W_O^i \in \mathbb{R}^{h \times h}$ 是投影矩阵。

MHA 块之后是前馈网络 (FFN), 它由两个完全连接的层组成, 在它们之间应用了非线性激活函数。它处理注意力输出 O^i 生成下一个解码器层的输入, 如下所示:

$$X^{i+1} = \sigma(O^i \cdot W_1^i) \cdot W_2^i, \quad (4)$$

其中 $W_1^i \in \mathbb{R}^{h \times d_{\text{FFN}}}$ 和 $W_2^i \in \mathbb{R}^{d_{\text{FFN}} \times h}$ 是两个线性层的权重矩阵, $\sigma(\cdot)$ 表示激活函数。

在解码阶段, 第 i 个解码器层接收单个 token $x^i \in \mathbb{R}^{b \times 1 \times h}$ 。通过将新计算的键值对与已有键值对拼接来更新 KV 缓存:

$$\begin{aligned} K^i &= \text{concat}(K^i, x^i \cdot W_K^i), \\ V^i &= \text{concat}(V^i, x^i \cdot W_V^i). \end{aligned} \quad (5)$$

解码阶段中剩余的注意力和前馈计算与预填充阶段中的相同。

3 方法

LLM 推理调度。我们的方法针对具有大型 KV 缓存的 LLM 推理系统, 这些缓存存储在 CPU DRAM 上并根据需要提取到 GPU 内存中。由于 LLM 有许多层和许多批次的输入需要处理, 因此有不同的调度策略来确定如何跨批次和层执行计算, 以优化特定的性能目

标, 例如最小化延迟或最大化吞吐量。逐行排程 (如附录所示) A.1 使用分层执行一次处理一批。在这种情况下, 只要可行, 模型权重就会保存在 GPU 内存中。如果模型权重也卸载到 CPU, 则 KV 缓存和单层的模型权重都会传输到 GPU, 针对当前批次进行处理, 然后清除。这个过程逐层重复, 直到生成令牌。当最大程度地减少延迟是主要目标时, 首选此方法, 因为批次中的所有提示都经过完全处理, 以在继续下一个批次之前生成完整的上下文。

逐列调度 (附录 A.1) 通过增加有效批量大小 (批次数量乘以批次大小) 并行处理更多序列, 但代价是更长的延迟。在这种方法中, 模型权重被卸载到 CPU 内存以适应大批量。单层的模型权重和 KV 缓存将传输到 GPU 内存并进行第一批处理。当前批次不会移至下一层, 而是使用同一层处理后续批次, 同时保持权重在 GPU 内存中固定。一旦第一层处理了一组批次, 每个批次的处理就会转移到第二层。请注意, 有效批量大小受到激活和 KV 缓存的可用存储的限制, 因为一旦超过 GPU 内存容量, 它们仍必须存储在 CPU 内存或外部存储中。

我们提出的设计独立于调度策略, 无论是逐行还是逐列, 旨在将大部分 PCIe 传输时间与 GPU 计算重叠, 从而提高整体效率。

3.1 设计概览

为了缓解 PCIe 压力并提高 GPU 计算利用率, 我们提出了一种新方法, KVPR, 重计算 GPU 上的部分 KV 缓存, 同时将其余 KV 缓存传输到 GPU。如图 2, KVPR 包括三个主要模块: 分析器、调度器和运行时。用户配置包括性能目标 (即延迟或吞吐量)、数据参数 (例如提示长度、生成长度、批量大小) 以及模型信息 (例如输入嵌入维度和层数)。**分析器模块**收集系统统计数据, 以刻画 PCIe 带宽、GPU 处理速度等硬件特性。例如, 分析器模块利用批量大小、模型信息和序列长度来表征 PCIe 带宽。结合这些信息和用户配置, **调度模块**通过求解线性规划问题来计算最优 KV 缓存重计算切

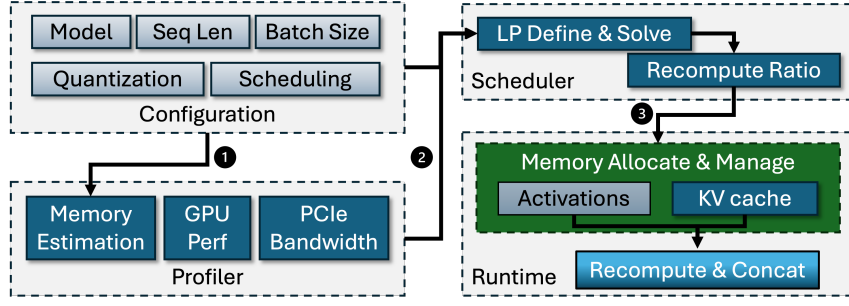


图 2: KVPR 的设计概览。用户配置和 profiler 信息输入 scheduler, scheduler 计算最佳 KV 缓存重计算比例; 随后 runtime 重叠数据传输和 GPU 计算以提升推理效率。

分点, 目标是在推理过程中最大化计算与通信的重叠, 以及 GPU 和 PCIe 带宽利用率。运行时模块则使用该执行策略处理用户输入, 并管理内存分配和数据传输流。

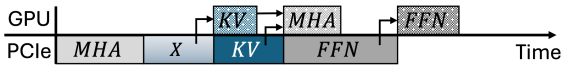
3.2 调度器模块

在本节中, 我们将描述如何 KVPR 适用于逐行或逐列计划。

具有 KV 缓存部分重计算的逐行调度。如果性能目标是最大化吞吐量, 则调度程序模块将启动逐行执行计划。逐行调度的朴素卸载管道如图所示 3(a), 其中 KV 缓存和模型权重都被卸载到 CPU 内存。所需数据通过 PCIe 异步传输到 GPU 以执行 MHA 和 FFN 块。为了简单起见, 图中省略了将新生成的 KV 对存储到 CPU 内存的过程。由于 KV 缓存的大小比 MHA 权重更大, 因此在异步传输过程中它会较晚到达 GPU。如果模型权重未卸载到 CPU, 则管道会略有不同。在这种情况下, 只有 MHA 块会等待 KV 缓存数据传输到 GPU 才开始计算。



(a) 带异步数据传输的逐行调度朴素卸载管道。GPU 和 PCIe 分别表示 GPU 计算和数据传输, 箭头表示数据依赖关系。



(b) 通过 KV 缓存部分重计算卸载用于行式调度的管道。

图 3: 两种卸载管道的比较。

在 KVPR, GPU 不是将整个 KV 缓存从 CPU 内存传输到 GPU 内存, 而是使用首先从 CPU 传输的相应输入激活重计算部分 KV 缓存, 而剩余的 KV 缓存则异步传输到 GPU, 如图所示 3 (二)。然后, GPU 将重计算的 KV 缓存与传输的 KV 缓存合并以执行 MHA 计算。**具有 KV 缓存部分重计算的逐列调度。**当性能目标是最大化吞吐量时, 调度程序模块采用逐列执行计划。这种方法, 如图所示 4, 通过跨多个批次重用模型权重来适应大批量推理。一旦批次 0 的 KV 缓存完全传输, 批次 1 的激活就会被传输。同时, GPU 开始计算批次 0 的 MHA。与逐行调度不同, 逐列调度在移至下一个批次之前按顺序处理单个批次内的所有层, 而逐列调度则在同一层上处理多个批次。因此, 必须存储与重计算的 KV 缓存相对应的激活, 直到该批次的生成完成。

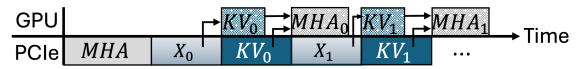


图 4: 通过 KV 缓存部分重计算卸载用于列调度的管道, 以最大限度地提高吞吐量。

确定最佳 KV 缓存分割点。在这两种调度方法中, 目标都是确定最佳分割点, 该分割点定义了 KV 缓存在 GPU 上重计算的部分和从 CPU 内存传输的部分之间的划分。该问题可以表述为线性规划问题。逐行调度可以视为逐列调度的特例, 其中不传输用于重计算 KV 缓存的激活。我们首先制定逐列计划的问题, 然后演示它如何简化为逐行计划。

给定当前序列长度 s' (大于提示长度 s)，传输到 GPU 的第 i 层激活表示为 $X^i[0:l]$ ，其中 $0 \leq l \leq s'$ 。后续 token 的剩余 KV 缓存表示为 $K^i[l:s']$ 和 $V^i[l:s']$ 。这些激活的内存使用量为：

$$\begin{aligned} M_{X^i[0:l]} &= b \times l \times h \times p, \\ M_{KV^i[l:s']} &= 2 \times b \times (s' - l) \times h \times p. \end{aligned} \quad (6)$$

重计算 KV 缓存 $X^i[0:l]$ 要求：

$$\begin{aligned} K^i[0:l] &= X^i[0:l] \cdot W_K^i, \\ V^i[0:l] &= X^i[0:l] \cdot W_V^i. \end{aligned} \quad (7)$$

GPU 上的重计算需要以下浮点运算

$$N_{KV^i[0:l]} = 4 \times b \times l \times h^2. \quad (8)$$

因此，重计算时间 t_{gpu}^i KV 缓存由下式给出

$$t_{recomp}^i = \frac{N_{KV^i[0:l]}}{v_{gpu}}, \quad (9)$$

其中 v_{gpu} 表示 GPU 处理速度。总处理时间 t^i 如下：

$$t^i = \frac{M_{X^i[0:l]}}{v_{com}} + \max\left(t_{recomp}^i, \frac{M_{KV^i[l:s']}}{v_{com}}\right), \quad (10)$$

其中 v_{com} 表示激活和 KV 缓存的数据传输速度。

目标是确定最优 l 最大限度地减少总处理时间 t^i ，这变成了一个线性规划问题：

$$\begin{aligned} \min_l \quad & t^i \\ \text{s.t.} \quad & 0 \leq l \leq s' \quad \forall i \in \{1, \dots, n\}. \end{aligned} \quad (11)$$

最佳分割点 l 取决于当前序列长度 s' ，它在生成过程中增加，因此必须自适应地确定。幸运的是，解决这一线性规划问题在计算上可以忽略不计，因为只有一个整数变量。若式中第一项 (10) 被省略，问题就简化为逐行调度。

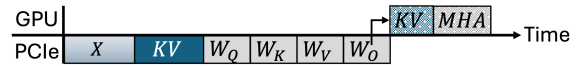
3.3 运行时模块

异步重叠。为了实现 GPU 计算和 CPU-GPU 通信的并发执行，运行时模块采用包含六类任务的通信并行策略：权重加载、KV 缓存加载、激活加载、重计算激活加载、KV 缓存存储和激活存储，详见附录 A.2。结合双缓

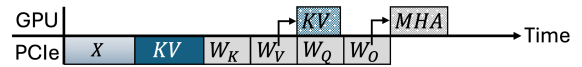
冲和预取技术，运行时在处理当前批次的同时，加载下一层权重，取回 KV 缓存重计算所需激活和下一批次的 KV 缓存，并存储上一批次的缓存和激活。

固定内存。与之前的工作一样，优化数据传输 (Sheng et al., 2023; Yu et al., 2024)，我们利用固定的 CPU 内存来重计算激活并将权重传输到 GPU。使用固定内存可以实现更快的异步传输，因为它避免了将数据分页进出的需要。

隐藏 KV 缓存部分重计算。如果 KV 缓存和模型权重都被卸载，且传输的 KV 缓存小于模型权重大小，那么带 KV 缓存部分重计算的粗粒度计算管道可能会降低推理性能。原因是重计算必须等待所有 MHA 权重 (W_Q, W_K, W_V 和 W_O) 加载完成，如图 5(a) 所示，这会延迟 MHA 计算。然而，KV 缓存重计算只需要 W_K 和 W_V (式 (7))，无需等待完整权重加载。为此，我们实现了细粒度 MHA 管道，优先加载 W_K 和 W_V 。一旦这些权重可用，KV 缓存重计算即可立即开始。如图 5(b) 所示， W_K 和 W_V 用于 KV 缓存部分重计算，随后 W_Q 和 W_O 用于 MHA 计算。这种方法有效地将 KV 缓存重计算与权重加载重叠，确保在最坏情况下性能不会低于受权重加载瓶颈限制的基线。



(a) 具有延迟 KV 缓存部分重计算的粗粒度卸载管道。



(b) 细粒度卸载管道与 KV 缓存重计算和权重加载重叠。

图 5: MHA 层不同粒度的卸载管道比较。

4 实验

硬件。在我们的实验中，我们使用具有 40 GB 内存的 NVIDIA A100 GPU，通过 PCIe 4.0 x16 接口连接到 CPU，提供 32 GB/s 的带宽。CPU 是 AMD EPYC 处理器，具有 64 核，运行频率为 2.6 GHz。我们的方法和实现自动适应底层硬件，从而允许跨不同系统架构灵活

部署。

模型。我们评估 KVPR 使用 OPT 模型 (Zhang et al., 2022) 参数大小范围从 6.7B 到 30B。虽然我们的实验侧重于 OPT 模型，但本工作中提出的重计算技术与其他 LLM 架构兼容，例如 LLaMa (Touvron et al., 2023) 和 GPT-3 (Brown et al., 2020)，由于它们相似的注意力机制 (Vaswani et al., 2017)。

工作量。我们评估 KVPR 针对两种类型的工作负载：面向延迟的和面向吞吐量的。在面向延迟的工作负载中，模型权重保留在 GPU 内存中，以避免代价高昂的重复加载。由于 GPU 内存大小有限，实验使用 OPT-6.7B 和 OPT-13B 进行。在面向吞吐量的工作负载中，模型权重在计算后卸载到 CPU，以释放更多 GPU 内存来处理更大的批次。使用 OPT-6.7B、OPT-13B 和 OPT-30B 评估此设置。

为了提供准确的比较，我们使用与 FlexGen 中相同的数据集 (Sheng et al., 2023) 提示统一填充为相同的长度，模型配置为每个提示生成 32 或 128 个标记。为了评估不同输入场景的性能，我们的评估使用 256、512 和 1024 个标记的提示长度。性能指标包括面向延迟的工作负载的解码延迟（生成令牌所需的时间）和面向吞吐量的工作负载的解码吞吐量（每秒生成的令牌），如下所示 KVPR 不影响预填充性能。我们分别报告五次测试运行的平均解码延迟和吞吐量。

基线。在我们的实验中，我们使用 DeepSpeed Inference (Aminabadi et al., 2022)，Hugging Face Accelerate (Gugger et al., 2022) 作为面向延迟的工作负载实验的基准，因为 Hugging Face Transformers 库目前支持 KV 缓存卸载到 CPU 内存，同时仍保留 GPU 内存中的模型权重。我们使用 FlexGen (Sheng et al., 2023) 作为面向吞吐量的工作负载实验的基准，因为它通过将模型权重和 KV 缓存卸载到 CPU 来支持逐列调度。

执行。KVPR 在 Hugging Face Transformers (v4.46.1) 之上实现 (Wolf et al., 2020) 和 Flex-

Gen (Sheng et al., 2023) 确保与基线进行公平比较的框架。在 Transformers 实现中，我们利用 GPU 内存中的双缓冲来重叠跨解码器层的 KV 缓存传输。对于 Transformers 和 FlexGen 实现，我们利用 CUDA 流来实现异步重叠，如算法中所述 1。

4.1 面向延迟的实验

我们评估不同序列长度设置完成单个批次所需的解码延迟。图 7 表明 KVPR 对于 OPT-6.7B 和 OPT-13B，其性能始终优于基准、DeepSpeed Inference 和 Hugging Face Accelerate。实验结果表明 KVPR 减少解码延迟，尤其是在较长的生成长度下。例如，OPT 6.7B，提示长度为 128，生成 128 个令牌，延迟减少约 35.8% 与 Hugging Face Accelerate 相比。附录中提供了详细的实验结果，包括 KV 缓存大小、GPU 峰值内存使用情况以及生成过程中的最佳重计算分割点 A.3 和 A.4。

4.2 面向吞吐量的实验

我们还评估解码阶段的吞吐量性能，如下所示 KVPR 不影响预填充阶段。为了最大化吞吐量，我们将有效批量大小设置为 32×8 ，这意味着每个层在移动到下一层之前按顺序计算 8 个大小为 32 的批量。图第一行 6 显示结果，证明 KVPR 在不同模型的所有序列长度设置下，其性能始终优于 FlexGen。最高可达 15.1%，46.2% 和 29.0% 分别提高了 OPT-6.7B、OPT-13B 和 OPT-30B 的吞吐量。附录中提供了低端 GPU 系统上的其他实验结果 A.5。

我们也比较 KVPR 使用 FlexGen，批量大小从 1 到 48 不等，固定提示长度为 1,024，生成长度为 32，如图第二行所示 6。KVPR 在所有批量大小上始终优于 FlexGen。随着 KV 缓存变大，KVPR 由于减少了 PCIe 总线上的 KV 缓存传输，显示出更大的性能优势。

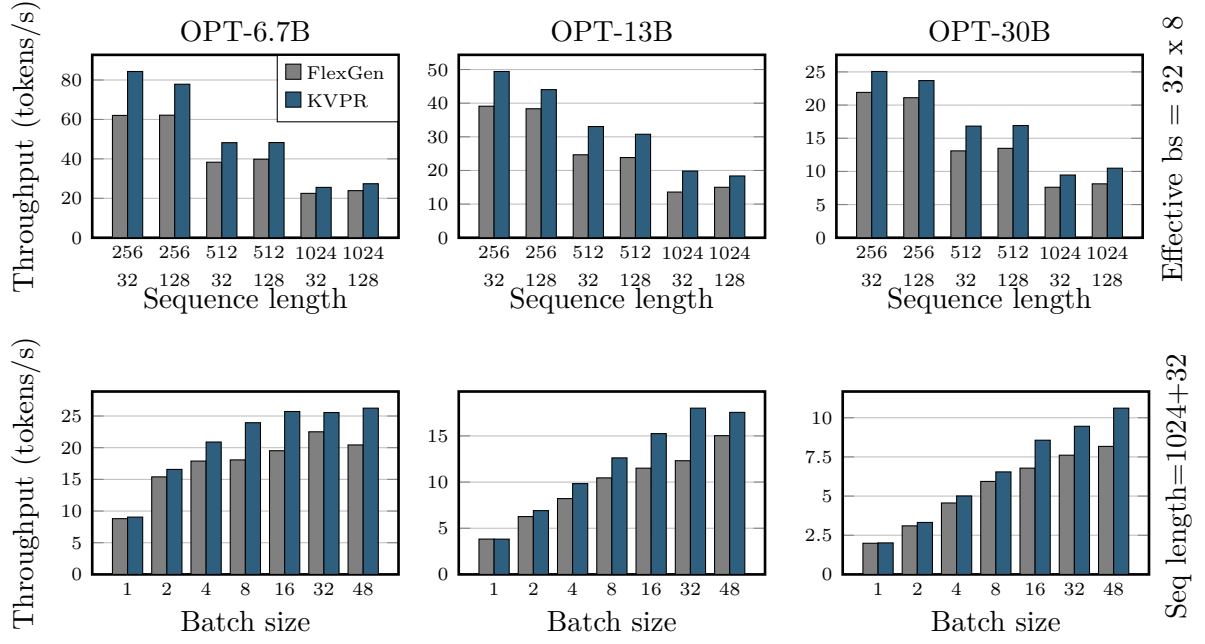


图 6: 各种型号和配置的吞吐量比较。

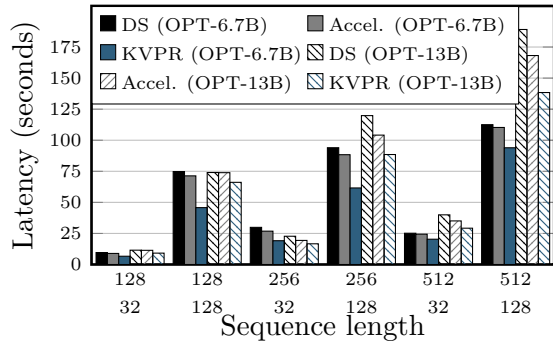


图 7: 批量大小为 64 时, 不同序列长度下单批次解码延迟。

4.3 GPU 利用率

为了评估效率的提高, 我们分析了时间资源利用率 KVPR 和 FlexGen 如图所示 8。首先在预填充阶段, 两种方法都达到了 GPU 的充分利用, 因为预填充阶段是计算密集型的。然而, 在解码阶段, 与 FlexGen 相比, KVPR 提高 GPU 利用率, 从 85 增加到% 至 99% 平均而言, 通过将 GPU 计算与 CPU-GPU 数据传输重叠, 同时保持黑线所示的相同峰值内存使用量。

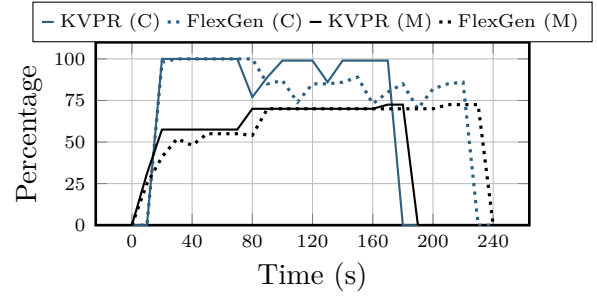


图 8: KVPR 与 FlexGen 在解码阶段的计算和内存资源使用情况。

4.4 KV 缓存压缩

我们应用分组 4 位量化来压缩 KV 缓存, 这已被证明对模型精度的影响最小 (Sheng et al., 2023)。图 9 表明应用压缩减少了传输到 GPU 的数据量, 从而进一步提高了解码吞吐量。这些结果展示了兼容性 KVPR KV 缓存压缩及其通过缓解 PCIe 带宽瓶颈实现额外性能提升的潜力。

4.5 消融研究

隐藏 KV 缓存部分重计算。为了评估将 KV 缓存重计算与权重加载重叠的细粒度卸载管道的有效性, 我们使用 OPT-6.7B 模型进行了实验。在此消融中, 我们使用较小的 KV 缓

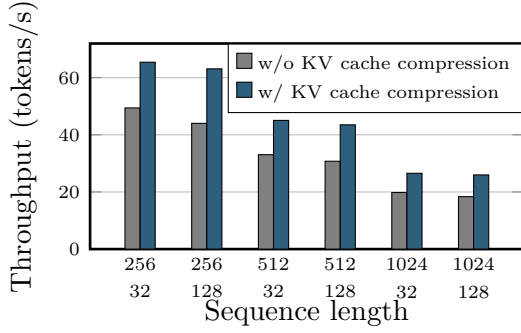


图 9: 通过在 OPT-13B 模型上启用 KV 缓存压缩来提高解码吞吐量。

存大小来确保 MHA 权重始终晚于 KV 缓存到达 GPU。表 2 展示了较小批量大小下的解码延迟，并比较三种配置：FlexGen、不隐藏 KV 缓存重计算的 KVPR，以及隐藏 KV 缓存重计算的 KVPR。当 batch size 为 1 且 KV 缓存最小时，FlexGen 可以优于不隐藏重计算的 KVPR。通过将 MHA 权重传输与 KV 缓存重计算重叠，KVPR 可确保在该场景下性能不低于 FlexGen，尤其是在权重加载是主要瓶颈时。该结果表明，KVPR 同时适用于小批量和大批量设置，从而为提升解码性能提供统一方法。

Batch size	1	2	4	8	16	32
KV cache (MB)	3	6	12	24	48	64
FlexGen	1.761	3.488	6.646	12.826	23.795	41.210
KVPR (w/o. hiding KV recomputation)	1.749	3.461	6.766	12.930	23.613	43.462
KVPR (w. hiding KV recomputation)	1.774	3.586	6.696	12.986	24.557	43.945

表 2: OPT-6.7B 模型的提示长度和生成长度分别为 256 和 64。每个 MHA 块 (W_Q , W_K , W_V , 和 W_O) 需要 128 MB 内存。

运行时细分。 图 10 给出了 KVPR 和 FlexGen 在解码阶段的 MHA 块运行时细分。KVPR 将 KV 缓存传输时间从 58% 显著降低到 38%，激活传输仅占总运行时间的 8%。通过基于传输的激活重计算部分 KV 缓存，GPU 计算时间从 2.3% 增加到 13.3%。这表明 KVPR 能有效重叠 GPU 计算与 CPU-GPU 通信，大幅减少 CPU 到 GPU 的数据传输量，缓解限制 LLM

推理性能的 PCIe 瓶颈。

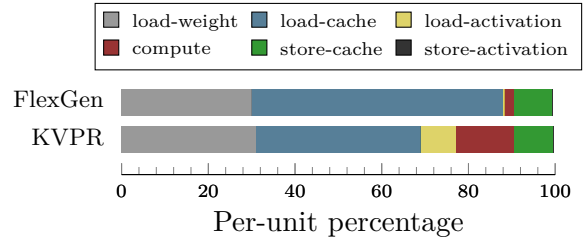


图 10: 运行时细分 KVPR 和 FlexGen。

5 相关工作

为满足资源受限环境中的 LLM 内存需求，卸载技术旨在尽量降低 CPU 与 GPU 之间的数据传输延迟。FlexGen (Sheng et al., 2023) 将权重、激活和 KV 缓存卸载到 CPU 内存或外部存储，并将优化表述为图遍历问题，以最大化大批量场景下的吞吐量。HeteGen (Zhao et al., 2024a) 使用 CPU 对卸载权重执行部分计算，同时将剩余工作负载交给 GPU。Twin-Pilots (Yu et al., 2024) 在算子层面进一步优化 CPU 与 GPU 之间的工作负载平衡。Fast-Decode (He and Zhai, 2024) 通过将 KV 缓存和注意力计算完全卸载到 CPU 来减少 KV 缓存数据移动。Park and Egger 和 NEO (Jiang et al., 2024) 将 GPU 线性投影计算与跨多个批次的 CPU 侧注意力计算重叠，以提高资源利用率。

ALISA (Zhao et al., 2024b) 基于稀疏性压缩 KV 缓存，并卸载超出 GPU 显存容量的 KV 缓存。在将 KV 缓存加载到 GPU 时，ALISA 会先重计算一部分 KV 缓存，再传输剩余部分；本文则通过自适应确定最佳切分点，使重计算和传输充分重叠。此外，ALISA 仅处理逐行调度，而 KVPR 扩展到了逐列调度。KVPR 与 CPU 辅助方法和 KV 缓存压缩方法正交，因此可以与这些技术结合，进一步提升整体系统性能。附录 A.7 的额外实验表明，在某些分布式系统配置中 CPU 可能成为瓶颈。相比之下，KVPR 优化 GPU 利用率和数据传输效率，无需依赖额外 CPU 资源，也不需要 KV 缓存

做近似。

6 结论

本文提出 KVPR, 一种高效的 CPU-GPU I/O 感知 LLM 推理方法, 用于加速 KV 缓存加载。KVPR 通过 KV 缓存部分重计算减少 CPU 与 GPU 之间的数据传输。通过将该重计算与数据传输重叠, KVPR 显著减少 GPU 空闲时间并提升整体推理性能。未来工作可以将该方法扩展到从远程网络存储加载 KV 缓存的场景, 或扩展到大规模多 GPU 基础设施, 从而进一步提升其适用性和性能。

7 局限性

我们的研究代表了通过利用 KV 缓存部分重计算来优化 LLM 推理效率的重要一步。然而, KVPR 有一定的局限性, 为未来的研究指明了方向。首先, 我们的方法目前仅限于单 GPU 和数据并行多 GPU 推理。它尚未扩展到高级分布式系统, 例如模型或张量并行性。将这种方法扩展到这些范例可以支持更大的模型尺寸。其次, 虽然我们解决了 CPU-GPU 通信中的 PCIe 带宽瓶颈, 但没有考虑从磁盘或网络存储加载 KV 缓存的场景。尽管如此, KVPR 仍可被调整用于加速此类设置中的预填充阶段。第三, 当前的实现仅在推理开始时执行系统分析, 并在整个过程中假设静态硬件条件。结合动态分析和运行时自适应优化可以增强该方法的稳健性和效率, 特别是在异构或多租户环境中。

8 致谢

我们衷心感谢所有审稿人的宝贵时间和建设性意见。本材料基于 NSF 奖项号 2224319、REAL@USC-Meta 中心和 VMware 礼品支持的工作。所表达的观点、意见和/或调查结果属于作者的观点、意见和/或调查结果, 不应被解释为代表美国政府的官方观点或政策。

References

- Reza Yazdani Aminabadi, Samyam Rajbhandari, Ammar Ahmad Awan, Cheng Li, Du Li, Elton Zheng, Olatunji Ruwase, Shaden Smith, Minjia Zhang, Jeff Rasley, et al. 2022. Deepspeed-inference: enabling efficient inference of transformer models at unprecedented scale. In *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 1–15. IEEE.
- Zhihao Bai, Zhen Zhang, Yibo Zhu, and Xin Jin. 2020. PipeSwitch: Fast pipelined context switching for deep learning applications. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 499–514.
- Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2024. Flashattention: fast and memory-efficient exact attention with io-awareness. In *Proceedings of the 36th International Conference on Neural Information Processing Systems*, NIPS '22.
- Tri Dao, Daniel Haziza, Francisco Massa, and Grigory Sizov. 2023. Flash decoding: Advances in efficient text generation.
- Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in llm serving with hcache. In *Proceedings of the Twentieth European Conference on Computer Systems*, pages 128–143.
- Google Gemini Team. 2024. Gemini: A family of highly capable multimodal models. Preprint, arXiv:2312.11805.
- Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM Conference on Recommender Systems*, RecSys '22, page 299–315.
- Sylvain Gugger, Lysandre Debut, Thomas Wolf, Philipp Schmid, Zachary Mueller, Sourab

- Mangrulkar, Marc Sun, and Benjamin Bossan. 2022. Accelerate: Training and inference at scale made simple, efficient and adaptable. <https://github.com/huggingface/accelerate>.
- Jiaao He and Jidong Zhai. 2024. Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines. *Preprint*, arXiv:2403.11421.
- Coleman Richard Charles Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Sophia Shao, Kurt Keutzer, and Amir Ghohami. 2024. KVQuant: Towards 10 million context length LLM inference with KV cache quantization. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Xuanlin Jiang, Yang Zhou, Shiyi Cao, Ion Stoica, and Minlan Yu. 2024. Neo: Saving gpu memory crisis with cpu offloading for online llm inference. *Preprint*, arXiv:2411.01142.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, page 611–626.
- Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: Efficient generative inference of large language models with dynamic KV cache management. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 155–172.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. Camel: Communicative agents for "mind" exploration of large language model society. *Preprint*, arXiv:2303.17760.
- Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024a. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, page 38–56.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. 2024b. KIVI: A tuning-free asymmetric 2bit quantization for KV cache. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 32332–32344.
- OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, and et al. 2024. Gpt-4 technical report. *Preprint*, arXiv:2303.08774.
- Daon Park and Bernhard Egger. 2024. Improving throughput-oriented llm inference with cpu computations. In *Proceedings of the 2024 International Conference on Parallel Architectures and Compilation Techniques*, pages 233–245.
- Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. Flexgen: high-throughput generative inference of large language models with a single gpu. In *Proceedings of the 40th International Conference on Machine Learning*, ICML'23.
- Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. Llama: Open and efficient foundation language models. *Preprint*, arXiv:2302.13971.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30.
- Zhongwei Wan, Xin Wang, Che Liu, Samiul Alam, Yu Zheng, Jiachen Liu, Zhongnan Qu, Shen Yan, Yi Zhu, Quanlu Zhang, Mosharaf Chowdhury, and Mi Zhang. 2024. Efficient large language models: A survey. *Transactions on Machine Learning Research*.
- Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. 2024. Efficient streaming language models with attention sinks. In *The Twelfth International Conference on Learning Representations*.

- Chengye Yu, Tianyu Wang, Zili Shao, Linjie Zhu, Xu Zhou, and Song Jiang. 2024. [Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference](#). In *Proceedings of the 17th ACM International Systems and Storage Conference*, SYSTOR '24, page 91–103.
- Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. [Orca: A distributed serving system for Transformer-Based generative models](#). In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 521–538.
- Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona Diab, Xian Li, Xi Victoria Lin, Todor Mihaylov, Myle Ott, Sam Shleifer, Kurt Shuster, Daniel Simig, Punit Singh Koura, Anjali Sridhar, Tianlu Wang, and Luke Zettlemoyer. 2022. [Opt: Open pre-trained transformer language models](#). *Preprint*, arXiv:2205.01068.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Re, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2o: Heavy-hitter oracle for efficient generative inference of large language models. In *Thirty-seventh Conference on Neural Information Processing Systems*.
- Xuanlei Zhao, Bin Jia, Haotian Zhou, Ziming Liu, Shenggan Cheng, and Yang You. 2024a. Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices. In *MLSys*.
- Youpeng Zhao, Di Wu, and Jun Wang. 2024b. [ALISA: Accelerating Large Language Model Inference via Sparsity-Aware KV Caching](#). In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pages 1005–1017.
- Wenhao Zhu, Hongyi Liu, Qingxiu Dong, Jingjing Xu, Shujian Huang, Lingpeng Kong, Jiajun Chen, and Lei Li. 2024. [Multilingual machine translation with large language models: Empirical results and analysis](#). In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 2765–2781.

A 附录

A.1 调度方法

图 11 说明了从三层模型生成 2 个标记的两个解码计划 (L_0 , L_1 , 和 L_2) 在解码阶段。图中 11(a), 逐行调度在移动到下一个批次之前跨所有层处理每个批次。相比之下, 图 11(b) 显示了逐列计划, 其中每一层在移动到下一层之前都被重用来处理一组批次。

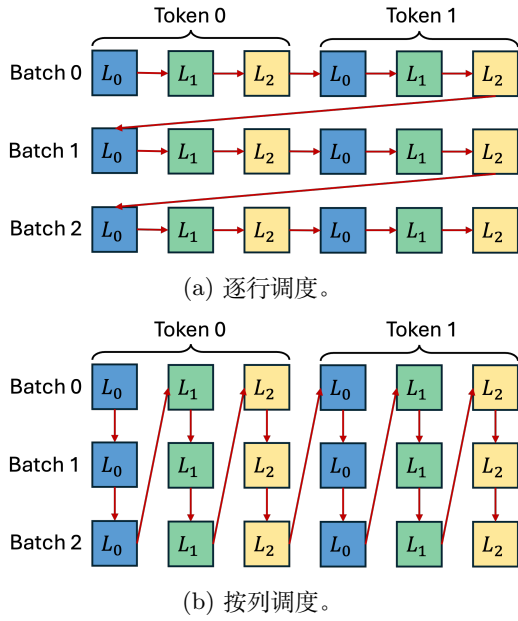


图 11: 两种不同的调度方法, 箭头表示调度顺序。

A.2 带重叠的 KV 缓存部分重计算

基于 FlexGen 构建 (Sheng et al., 2023) 的计算和通信重叠技术, 我们对其进行调整以支持 KV 缓存部分重计算。算法 1 可以同时执行最内层循环中的任务, 包括加载下一层的权重、加载 KV 缓存重计算的激活、重计算部分 KV 缓存、加载剩余的 KV 缓存和下一批的激活、存储上一批的 KV 缓存和激活以及执行当前批次的计算。虽然该算法是为逐列调度而设计的, 但单批的逐行调度是它的一个特例。

A.3 详细实验结果

表 3 和 4 展示了使用 OPT-6.7B 和 OPT-13B 的面向延迟的工作负载的详细实验结果。结果显示了之间的性能差异 KVPR 以及各种

Algorithm 1 带重叠的 KV 缓存部分重计算

```

for  $i = 1$  to generation_length do
  for  $j = 1$  to num_layers do
    for  $k = 1$  to num_GPU_batches do
      // Load the weight of the next layer
      load_weight( $i, j + 1, k$ )
      // Load the activation for KV cache
      recomputation of the next batch
      load_activation_recompute( $i, j, k + 1$ )
      // Load the KV cache and activation
      of the next batch
      load_cache( $i, j, k + 1$ )
      load_activation( $i, j, k + 1$ )
      // Compute this batch
      compute( $i, j, k$ )
      // Store the KV cache and activation
      of the previous batch
      store_activation( $i, j, k - 1$ )
      store_cache( $i, j, k - 1$ )
      // Synchronize all devices
      synchronize()
    end for
  end for
end for

```

配置的 GPU 峰值内存、解码延迟和吞吐量方面的基线 (带有 KV 缓存卸载的 Hugging Face Transformer)。尤其, KVPR 始终如一地实现较低的延迟, 同时保持可比较的内存使用量。

A.4 最优 KV 缓存切分点

图 12 给出最优的 KV 缓存分割点 l , 通过求解式中定义的线性规划问题得到 (11), 用于第 1 节中面向延迟的工作负载实验的第一个设置 4 (提示长度为 128, 生成长度为 32)。基于系统分析统计和 KV 缓存大小, 最佳分割点 l 当生成长度为 1 时, 为 182, 并且 l 当生成长度为 32 时, 增加到 128。

Method	Batch size	Prompt length	Generation length	Cache size (GB)	GPU peak mem (GB)	Decode latency (sec)	Decode throughput (tokens/s)
Accel.	64	128	32	5.0	14.427	8.905	222.788
	64	128	128	8.0	14.708	71.327	113.954
	64	256	32	9.0	16.337	26.825	73.961
	64	256	128	12.0	16.618	88.354	91.993
	64	512	32	17.0	20.154	24.390	81.344
	64	512	128	20.0	20.576	110.277	73.705
KVPR	64	128	32	5.0	14.364	6.651	298.284
	64	128	128	8.0	14.645	45.766	177.598
	64	256	32	9.0	16.212	19.138	103.666
	64	256	128	12.0	16.493	61.597	131.955
	64	512	32	17.0	19.904	20.349	97.501
	64	512	128	20.0	20.951	93.932	86.531

表 3: 对应图 7 的 OPT-6.7B 详细实验结果。

Method	Batch size	Prompt length	Generation length	Cache size (GB)	GPU peak mem (GB)	Decode latency (sec)	Decode throughput (tokens/s)
Accel.	64	128	32	7.812	26.083	11.409	173.891
	64	128	128	12.500	26.434	73.896	109.993
	64	256	32	14.062	28.087	19.381	102.368
	64	256	128	18.750	28.439	104.115	78.068
	64	512	32	26.562	32.851	35.066	56.579
	64	512	128	31.250	34.146	168.155	48.336
KVPR	64	128	32	7.812	26.005	9.148	216.867
	64	128	128	12.500	26.356	66.119	122.929
	64	256	32	14.062	27.931	16.654	119.127
	64	256	128	18.750	28.337	88.492	91.850
	64	512	32	26.562	33.203	29.215	67.911
	64	512	128	31.250	34.615	138.377	58.738

表 4: 对应图 7 的 OPT-13B 详细实验结果。

A.5 低端 GPU 上的系统性能

为了进一步证明其适应性 KVPR, 我们在低端系统上对其进行评估, 该系统配备 AMD EPYC 32 核 CPU 和 NVIDIA Quadro RTX 5000 GPU (16 GB HBM, 89.2 TFLOPS FP16 峰值性能), 通过 PCIe 4.0 x8 (16 GB/s 带宽) 连接。此系统设置中的 GPU TFLOPS、GPU 内存和 PCIe 带宽低于我们之前使用的默认系统中的值。尽管 GPU 速度和带宽降低了, KVPR 达到 15% 在相同的面向吞吐量的工作负载中, OPT-6.7B 的吞吐量高于 FlexGen, 如表所示 5。

Seq len	256/32	256/128	512/32	512/128	1024/32	1024/128
FlexGen	50.057	46.779	29.614	28.650	15.778	16.194
KVPR	53.976	49.860	33.666	32.277	18.285	18.108

表 5: 低端 GPU 系统上的吞吐量 (令牌/秒) 比较。

A.6 LLaMa 模型上的补充实验结果

除了小节讨论的 OPT 模式之外 4, 我们对最新的 LLaMa2-7B 和 LLaMa2-13B 模型进行了进一步的实验。使用相同的实验设置, 我们测量了在不同的提示和生成长度下处理单批大小为 64 的解码吞吐量。如图 13, KVPR 在两个 LLaMa2 模型上始终实现比基准 (DeepSpeed Inference 和 Hugging Face Accelerate) 更高的

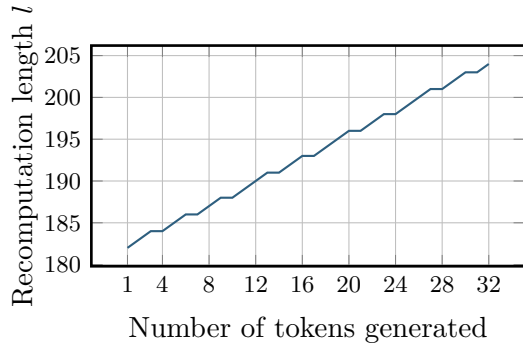


图 12: 生成过程中最优 KV 缓存分割点 l 的变化。

吞吐量。

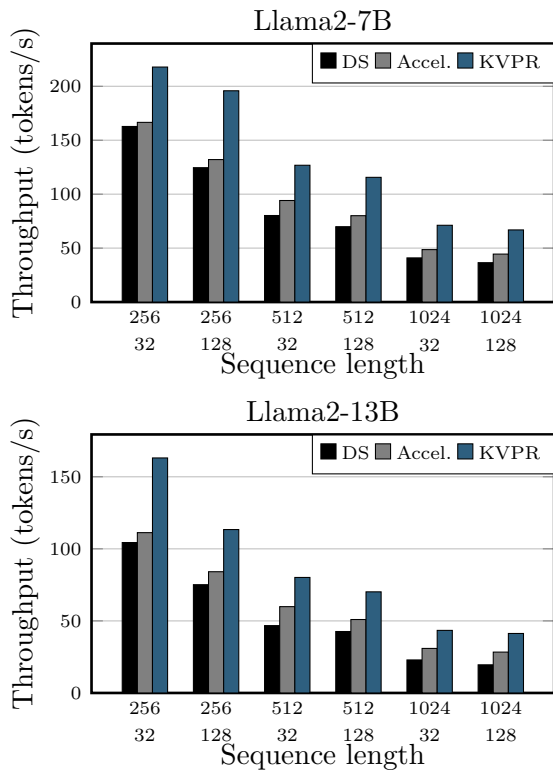


图 13: 跨不同序列长度的单个批量大小为 64 的解码吞吐量。

A.7 分布式系统设置与 CPU 辅助方法的比较

在本实验中，我们比较了 CPU 辅助卸载方法 FastDecode 的性能 (He and Zhai, 2024)，和 KVPR 在配备 8 个 NVIDIA A100 GPU 和一个 CPU 的 GPU 节点上，这是相同的 AMD EPYC 处理器 (64 核, PCIe 4.0 128 通道)，如第 1 节中所述 4。

我们运行 FastDecode 的多个并发进程 KVPR 在可用的 GPU 上，每个 GPU 专用于一个进程。此设置模拟多个用户共享单个计算节点或单个用户执行数据并行推理的场景。FastDecode 依赖 CPU 进行注意力计算，当管理多个并发推理进程时，CPU 成为瓶颈，导致性能下降。相比之下，KVPR 完全消除了对 CPU 的依赖，并优化了 PCIe 总线上的数据传输。

图 14 表明虽然 FastDecode 的吞吐量随着进程数量的增加而显著下降，KVPR 表现出更好的可扩展性，在单 CPU 和多 GPU 的系统中保持稳定的性能。

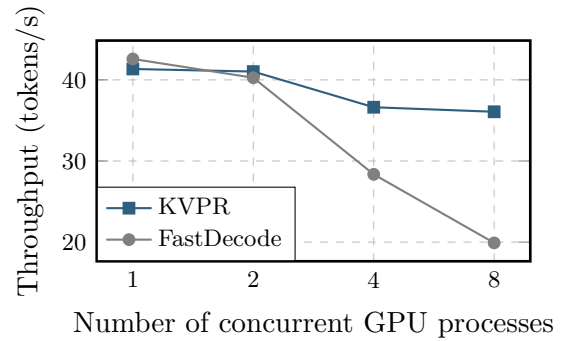


图 14: KVPR 与 FastDecode 在不同 GPU 工作负载下的吞吐量比较。

A.8 扩展相关工作

GPU 高效的 LLM 推理。最大化 GPU 利用率对于高效服务 LLM 以实现低延迟和高吞吐量至关重要。Orca (Yu et al., 2022) 采用迭代级调度来处理具有不同输出长度的批次，立即返回完成的序列以服务新的序列。PagedAttention (Kwon et al., 2023) 观察到 KV 缓存会随 token 生成动态增长和收缩，且序列生命周期和长度并不能预先确定。它通过将 KV 缓存管理为非连续内存块来解决这一问题。FlashAttention (Dao et al., 2024) 将注意力操作融合到单个内核中，并将 QKV 矩阵分割为更小块，以优化 GPU SRAM 使用并减少 HBM 访问开销，而我们的工作主要关注 PCIe 带宽优化。DeepSpeed Inference (Aminabadi et al., 2022)

结合 GPU 内存以及 CPU/NVMe 混合推理技术，增强了密集和稀疏 Transformer 模型的多 GPU 推理。Flash Decoding (Dao et al., 2023) 通过将键和值切分为更小块，实现并行注意力计算并组合最终输出，加速长上下文推理。HCache (Gao et al., 2025) 关注在在线推理中跨用户请求恢复并复用上下文状态，在延迟、容量和持久性之间取得平衡，以实现稳健的 LLM 服务。

KV 缓存优化。高效的 KV 缓存管理通过压缩或驱逐策略增强推理性能。KIVI (Liu et al., 2024b) 引入了免调整 2 位量化方法来压缩每个通道的键缓存和每个令牌的值缓存。同样，KVQuant (Hooper et al., 2024) 通过将每通道量化与 LLaMA 的预旋转位置嵌入量化相结合，应用 3 位压缩。对于驱逐，H2O (Zhang et al., 2023) 将 KV 缓存驱逐制定为动态子模块问题，优先考虑关键和最近的令牌以提高吞吐量。StreamingLLM (Xiao et al., 2024) 使用窗口注意力和固定大小的滑动窗口来保留最新的 KV 缓存，一旦缓存达到容量，就保持恒定的内存使用和解码速度。InfiniGen (Lee et al., 2024) 将低等级键缓存存储在 GPU 内存中，将值缓存卸载到 CPU，并根据近似注意力分数选择性地检索重要值。